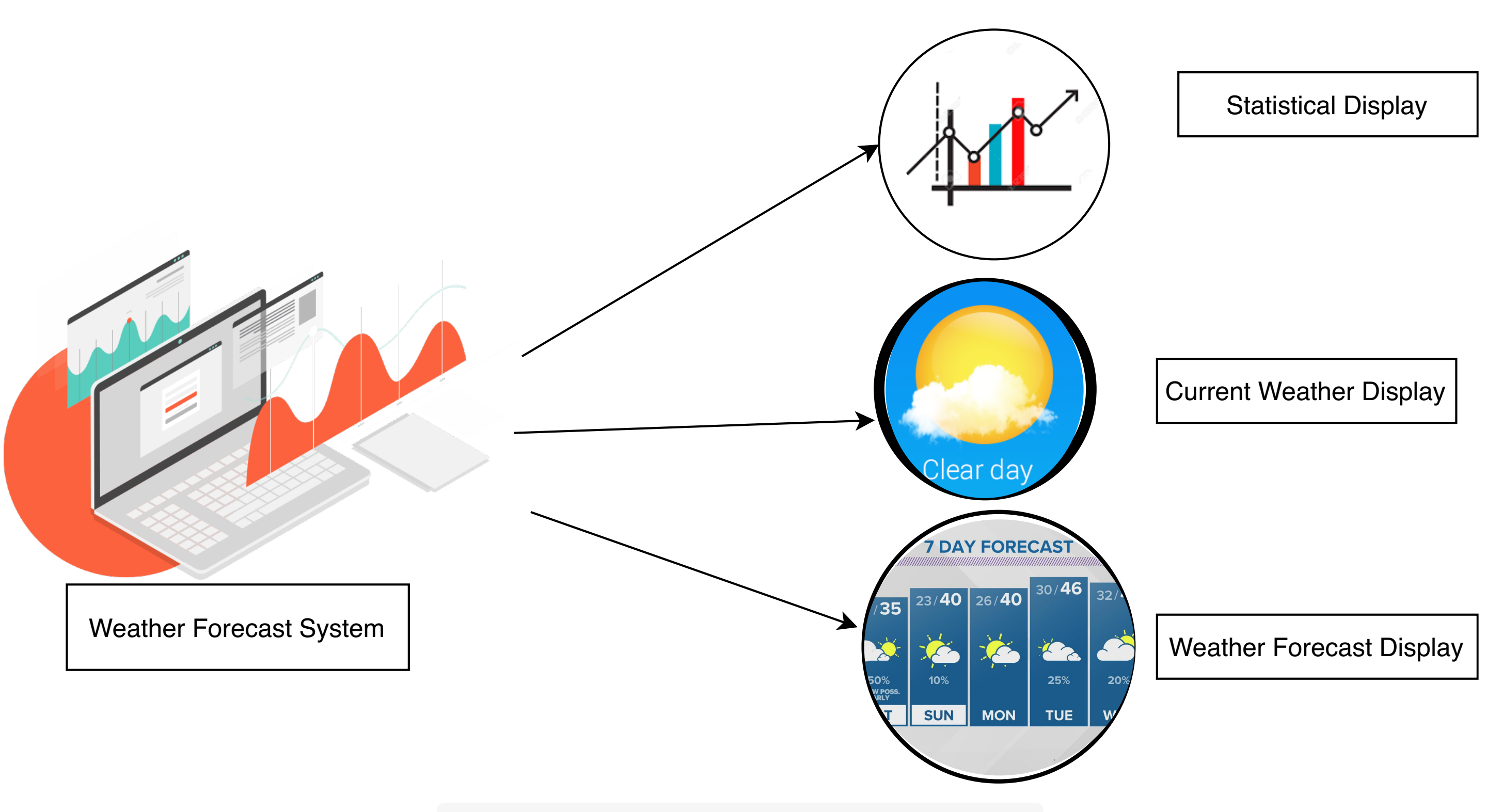
Observer Pattern

The Observer pattern defines a one-to-many relationship between objects. When the state of one object (the subject or observable) changes, all its dependents (observers) are automatically notified and updated. This pattern encourages loose coupling between the subject and its observers and is especially helpful for implementing distributed event-handling systems.

Imagine a weather station that gathers data and has various display elements that must show this data, such as a current condition display, a statistics display, and a forecast display. All these displays need to be updated in the event that the weather data changes. A direct approach would require a close coupling between every display element and the weather station, making the system less scalable, rigid, and more difficult to extend or maintain. The image below illustrates one such weather forecast system.



Observer Pattern - Weather Forecast System

To tackle this issue, the Observer pattern permits the weather station to release updates (observable) and the display elements to subscribe to these updates (observers). All subscribed displays receive automatic updates from the weather station whenever there is a change in the weather data. This method separates the weather station from the displays, increasing the system's flexibility and ease of further development.

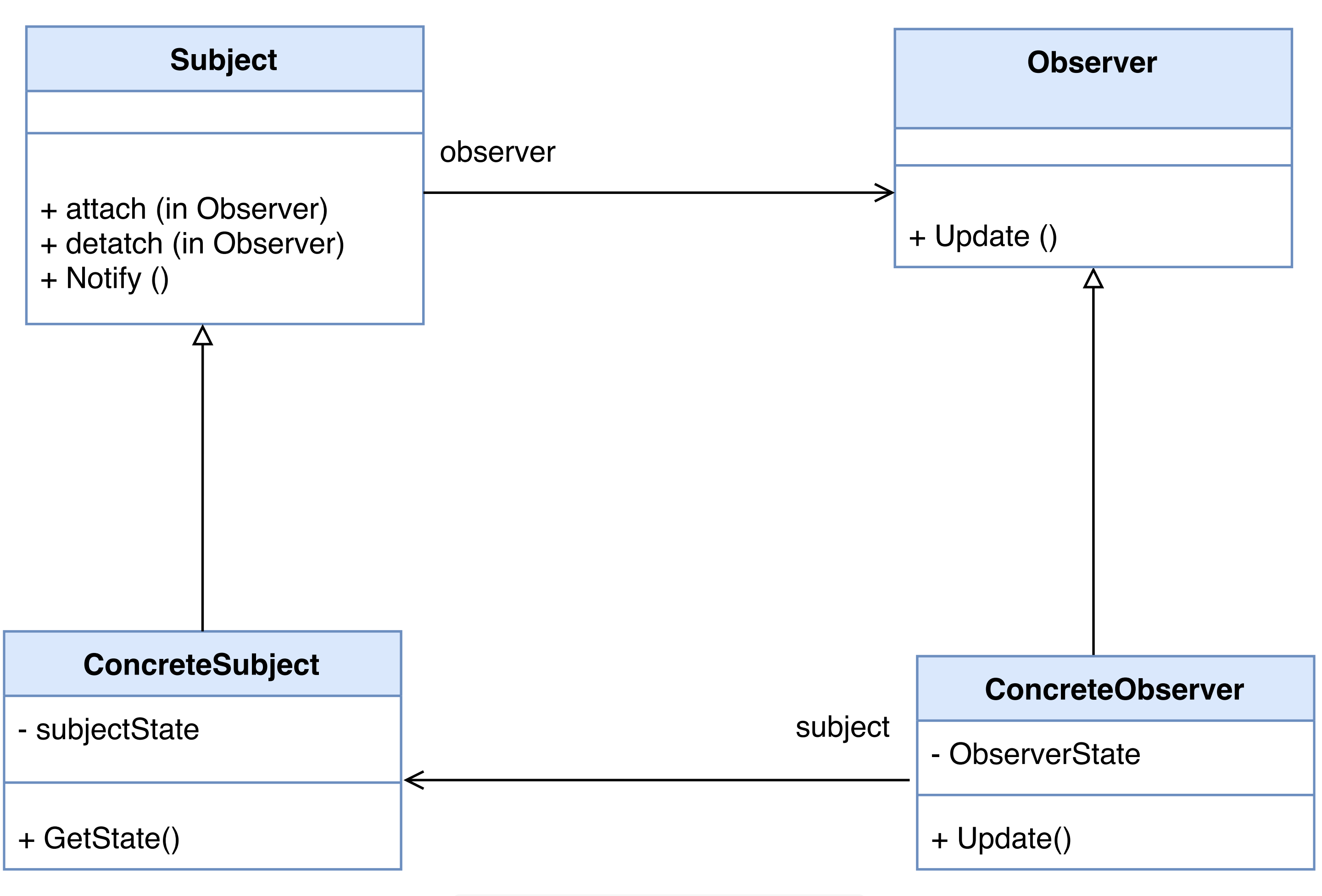
Real-World Example

A real-world example of the Observer pattern is seen in flight status monitoring systems at airports. In the flight status monitoring system, the Observer pattern is exemplified through the interaction between the flight status tracking system and the passengers or airline staff. The system, functioning as the 'observable', keeps an eye on each flight's status. Passengers and staff, acting as 'observers', sign up to receive updates on specific flights. When there's a change in a flight's status, like a delay or boarding announcement, the system automatically informs all subscribed observers about this change. This setup eliminates the need for passengers and staff to continuously check for updates manually. They're automatically kept in the loop, receiving real-time notifications about the flights they are interested in, mirroring the essence of the Observer pattern where changes in one object (the flight status) automatically trigger updates to all its dependents (the passengers and staff).

Structure of Observer Pattern

The Observer pattern's class diagram involves two primary types of components: the Observables (also known as Subjects) and the Observers. Here's a breakdown of the structure, along with a description of each component:

1. **Subject** (**Observable**):
   - **Interface or Abstract Class:** Typically, an interface or abstract class defines methods for attaching, detaching, and notifying observers.
   - Methods:
     - `attach(Observer)` : Registers an observer to the subject.
     - `detach(Observer)` : Removes an observer from the subject.
     - `notify()` : Notifies all registered observers when a change occurs.
2. **Concrete Subject**:
   - **Class:** Inherits or implements the Subject interface.
   - **State:** Maintains the state that should be consistent with the observers' state.
   - **Behavior:** When its state changes, it triggers the `notify()` method to update all its observers.
3. **Observer**:
   - **Interface or Abstract Class:** Defines the `update()` method that is called when the Subject changes.
   - Methods:
     - `update()` : The method through which observers receive updates from the subject.
4. **Concrete Observer**:
   - **Class:** Implements the Observer interface.
   - **State:** Maintains state that should stay consistent with the subject's.
   - **Behavior:** Implements the `update()` method to react to changes notified by the subject.



Observer Pattern - Class Diagram

Implementation of Observer Pattern

To implement the Observer Pattern, recap the weather station example discussed earlier. In this example, the weather station is the Subject, and the various displays, like current conditions and forecast displays, are the Observers.

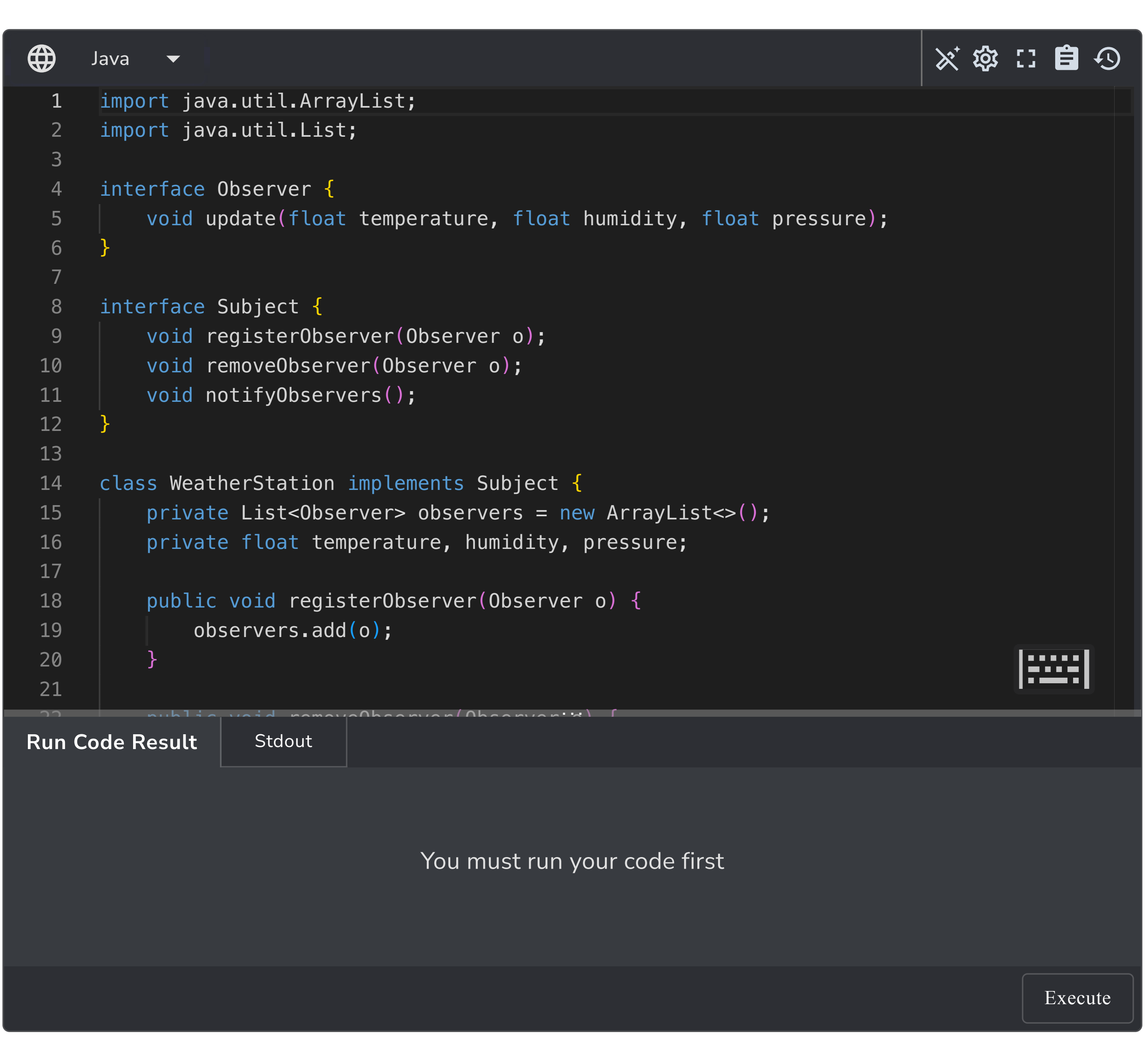
Let's look at the pseudocode of this example and then will go through the implementation in multiple languages:

```
Interface Observer {
    Method update(temperature: Float, humidity: Float, pressure: Float)
}

Interface Subject {
    Method registerObserver(observer: Observer)
    Method removeObserver(observer: Observer)
    Method notifyObservers()
}

Class WeatherStation implements Subject {
    Private observers: List of Observer
    Private temperature: Float
    Private humidity: Float
    Private pressure: Float

    Method registerObserver(observer: Observer) {
        observers.add(observer)
    }

    Method removeObserver(observer: Observer) {
        observers.remove(observer)
    }

    Method notifyObservers() {
        For each observer in observers {
            observer.update(temperature, humidity, pressure)
        }
    }

    Method setMeasurements(temperature: Float, humidity: Float, pressure: Float) {
        this.temperature = temperature
        this.humidity = humidity
        this.pressure = pressure
        notifyObservers()
    }
}

Class CurrentConditionsDisplay implements Observer {
    Private subject: Subject

    Constructor(subject: Subject) {
        this.subject = subject
        subject.registerObserver(this)
    }

    Method update(temperature: Float, humidity: Float, pressure: Float) {
        // Display the current conditions
    }
}

// Similar class for ForecastDisplay

// Client Code
Main {
    weatherStation = new WeatherStation()
    currentDisplay = new CurrentConditionsDisplay(weatherStation)
    forecastDisplay = new ForecastDisplay(weatherStation)

    weatherStation.setMeasurements(30.0, 65, 1013.1)
}
```

- **WeatherStation** (**Subject**): This is the entity that keeps track of weather information and alerts observers to any changes. It implements methods for adding, removing, and notifying observers.
- **Observer** (**Interface**): This interface is implemented by the `CurrentConditionsDisplay` and other displays. The update method of each observer is called when the state of the subject changes.
- **Observer Registration**: Display objects must register with the WeatherStation in order to receive updates.
- **Notification and State Changes**: When `WeatherStation` receives new measurements (`setMeasurements`), it calls `notifyObservers` , causing each registered display to update.

With this configuration, WeatherStation can send updates to every display that has registered without being closely connected to any of them. Adding and removing observers dynamically is possible, and adding new kinds of observers is possible without modifying the code of the subject.

Implementation



```java
import java.util.ArrayList;
import java.util.List;

interface Observer {
    void update(float temperature, float humidity, float pressure);
}

interface Subject {
    void registerObserver(Observer o);
    void removeObserver(Observer o);
    void notifyObservers();
}

class WeatherStation implements Subject {
    private List<Observer> observers = new ArrayList<>();
    private float temperature, humidity, pressure;

    public void registerObserver(Observer o) {
        observers.add(o);
    }

```

You must run your code first

Applications of Observer Pattern

1. **Event Handling Systems:** It helps handling of events in Graphical User Interfaces like clicks, keypresses, etc.
2. **Data Monitoring:** Monitoring and reacting to changes in data, such as stock market monitoring systems.
3. **Publish-Subscribe Systems:** It helps in implementing pub-sub systems where publishers emit events and subscribers react to them.
4. **Model-View-Controller** (**MVC**) **Frameworks**: Separating the model (data) from the view (UI) where the view reacts to changes in the model.
5. **Notification Services:** Pushing notifications in response to events or data changes, commonly seen in social media and news apps.

Pros and Cons

Pros	Cons
Loose Coupling Subjects and observers are loosely coupled, promoting modularity and reusability.	**Overhead** Maintaining a large number of observers for a subject can increase overhead.
Dynamic Subscription Observers can be added or removed at runtime, providing flexibility.	**Unexpected Updates** Observers might receive updates at times they aren't expecting, leading to potential issues.
Broadcast Communication Allows for broadcast-style communication, where subjects notify multiple observers.	**Memory Leaks** Can lead to memory leaks if observers aren't properly deregistered, especially in languages without automatic garbage collection.
Scalability Facilitates scalability in features like event handling and data monitoring.	**Complexity** Can make the system more complex and harder to debug due to indirect flow of control.

The Observer pattern is very useful when a system component needs to notify others about changes in its state or specific events. It encourages adaptability, decoupling, and effective communication. But in order to prevent problems with unexpected updates, performance overhead, and possible memory leaks, especially in systems with multiple observers or frequent state changes, careful handling is needed.

← Previous
Memento Pattern
Next →
State Pattern
☐ Mark as Completed